ESTRUCTURAS DE DATOS Y ALGORITMOS
(CDS-0202).

ARREGLOS ESTÁTICOS Y MATRICES

CARRERA DE DESAROLLO DE SOFTWARE

Nayeli Jamileth Córdova Pionce
https://github.com/Nayecordova/EDA-2N-ESPAM-NC.git

1 de noviembre del 2026

OBJETIVO

- Validar el dominio de las estructuras de datos lineales fundamentales (arreglos, matrices, listas enlazadas, pilas y colas circulares) aplicándolas de forma práctica a un escenario del mundo real.
- Explicar por qué una estructura específica (en este caso, los arreglos y matrices bidimensionales) resuelve de manera óptima las necesidades de mapeo espacial y control de asientos en tiempo real.

ANÁLISIS DEL PROBLEMA

a) Por qué la estructura asignada es la más adecuada para su caso.

Una sala de cine tiene filas y asientos fijos, y el sistema debe consultar y reservar en tiempo real. Elegimos una matriz (arreglo estático 2D) porque el tamaño de la sala es conocido y no cambia. Al estar en memoria contigua, accedemos a cualquier asiento con asientos [fila][columna] en O(1), sin recorrer nodos y sin el sobrecosto de punteros de una lista enlazada. Además, la matriz refleja directamente la distribución física de la sala.

b) La complejidad temporal en Big-O de sus operaciones principales.

Para el Sistema de Asientos y Mapeo en Tiempo Real de un auditorio o cine, las operaciones principales de consulta y actualización del estado de un asiento (disponible, reservado u ocupado) se diseñan sobre una estructura basada en arreglos en memoria contigua. Esto permite que tanto la lectura como la modificación del mapa de asientos se ejecuten con una complejidad temporal O(1) en tiempo constante, permitiendo respuestas inmediatas ante la alta concurrencia de usuarios.

Nuestra clave de defensa se define en que usamos acceso instantáneo en O(1) por fila, memoria contigua y cero sobrecosto de punteros. Básicamente, como los asientos están guardados juntos en la memoria de la compu (en un bloque continuo), el sistema no tiene que ponerse a buscar punteros por todos lados ni perder tiempo adivinando dónde está cada puesto; llega directo al asiento que necesita al instante y sin gastar memoria de más.

Para resolver el Sistema de Asientos y Mapeo en Tiempo Real para Cine, la estructura de Arreglos/Matrices Bidimensionales es la idónea ya que una matriz de 2 dimensiones (Fila \times Columna) representa de forma perfecta y directa la distribución física de las butacas de un auditorio, permitiendo un acceso directo por índices en tiempo constante O(1) para reservar o liberar un asiento.

Técnica SCQA (Situación, Complicación, Pregunta, Respuesta):

- **Situación:** Un auditorio o cine necesita gestionar la ocupación de cientos de asientos en tiempo real para evitar sobreventa o duplicidad de localidades.
- **Complicación:** Los métodos de búsqueda lineal o estructuras desorganizadas generan demoras y errores críticos en la taquilla o en la visualización web.
- **Pregunta:** ¿Cómo modelar la disposición espacial de las butacas garantizando un rendimiento óptimo de consulta y actualización en memoria?
- **Respuesta:** Implementar una matriz estática en Java donde cada celda almacene el estado del asiento (Libre, Ocupado, Reservado), logrando operaciones en orden O(1).
- **Verificación:** Captura de pantalla de la sección de análisis y la técnica SCQA lista para el documento PDF.



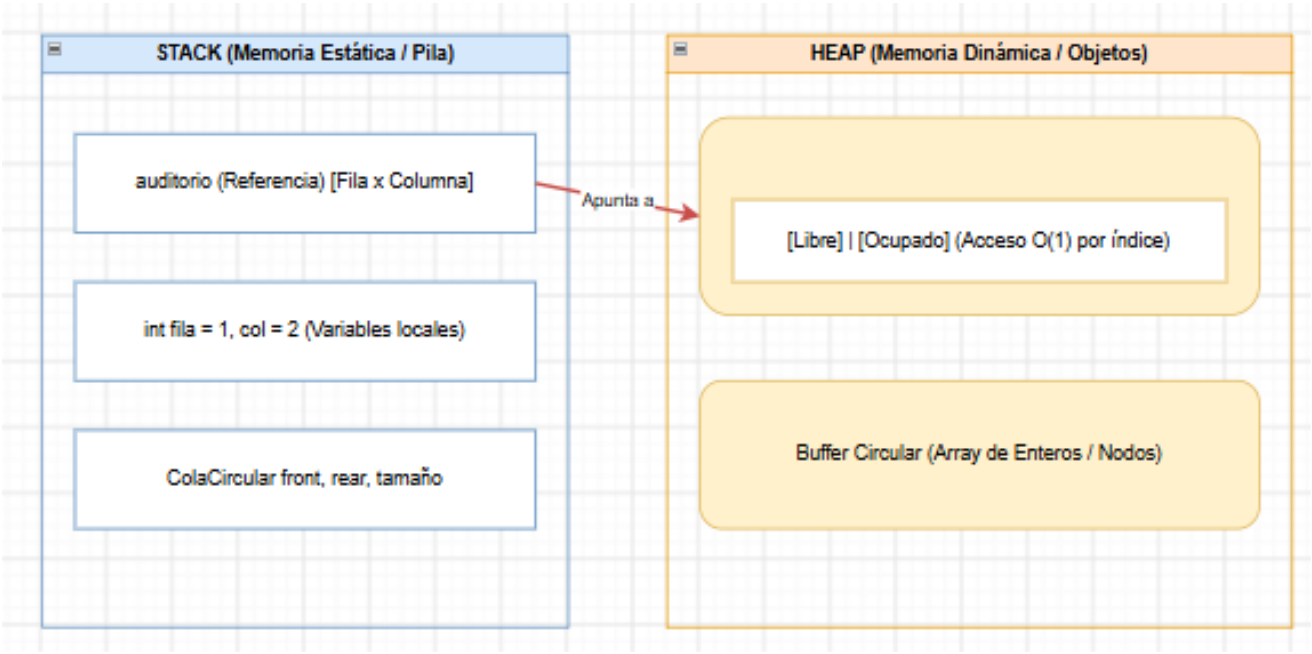
PARTICIPACIÓN EN EL FORO

Re: Grupo 2_Caso: Recepción Dinámica de Solicitudes y Trámites en Ventanilla Virtual

de NAYELI JAMILETH CORDOVA PIONCE - miércoles, 21 de octubre de 2026, 18:58

La estrategia consiste en estructurar los turnos o ventanillas con un tamaño máximo fijo predefinido (por ejemplo, un arreglo estático que funcione como una cola circular para los tickets, o una matriz donde las filas representen las ventanillas de atención y las columnas los turnos asignados).
Las listas enlazadas gastan memoria extra por cada nodo guardando el puntero o enlace al siguiente elemento (siguiente). Con un arreglo estático, el 100% de la memoria se usa puramente para los datos de la solicitud.

DIAGRAMAS DE ARQUITECTURA Y MEMORIA



DESARROLLO DE CÓDIGO JAVA

Este archivo implementa la matriz bidimensional para el sistema de asientos del auditorio

CasoForo.java

```java
public class CasoForo {
    private void inicializarAuditorio() {
        for (int i = 0; i < filas; i++) {
            for (int j = 0; j < columnas; j++) {
                auditorio[i][j] = "[Libre]"; // O(1)
            }
        }
    }

    public void reservarAsiento(int fila, int col) {
        if (fila >= 0 && fila < filas && col >= 0 &&
            if (auditorio[fila][col].equals("[Libre]")) {
                auditorio[fila][col] = "[Ocupado]";
                System.out.println("Asiento " + fila
            } else {
                System.out.println("El asiento ya est
            }
        } else {
            System.out.println("Coordenadas fuera de
        }
    }

    public void mostrarMapa() {
        System.out.println("--- MAPA DEL AUDITORIO --
        for (int i = 0; i < filas; i++) {
            for (int j = 0; j < columnas; j++) {
                System.out.print(auditorio[i][j] + "
            }
```

Implementación de nodos en memoria dinámica Heap y enlaces de referencias

```java
// ListaEnlazadaNodo.java > ListaEnlazadaSimple<T>
class Nodo<T> {
    T dato;
    Nodo<T> siguiente;

    public Nodo(T dato) {
        this.dato = dato;
        this.siguiente = null; // Referencia en memo
    }
}

class ListaEnlazadaSimple<T> {
    private Nodo<T> cabeza;

    public void insertar(T dato) {
        Nodo<T> nuevo = new Nodo<>(dato);
        if (cabeza == null) {
            cabeza = nuevo;
        } else {
            Nodo<T> actual = cabeza;
            while (actual.siguiente != null) {
                actual = actual.siguiente;
            }
            actual.siguiente = nuevo;
        }
    }
}
```

Métodos push, pop, peek y verificación de desbordamiento (overflow/underflow).

```java
// CasoForo.java | ListaEnlazadaNodo.java | PilaLIFO.java
// PilaLIFO.java > ...
public class PilaLIFO {
    private int[] datos;
    private int cima;
    private int capacidad;

    public PilaLIFO(int capacidad) {
        this.capacidad = capacidad;
        datos = new int[capacidad];
        cima = -1; // Underflow inicial
    }

    public void push(int valor) {
        if (cima == capacidad - 1) {
            System.out.println("Desbordamiento (Over
            return;
        }
        datos[++cima] = valor;
    }

    public int pop() {
        if (cima == -1) {
            System.out.println("Subdesbordamiento (U
            return -1;
        }
        return datos[cima--];
    }
```

Avance modular con la fórmula (rear + 1) % capacidad en O(1)

```java
// ColaCircular.java > ColaCircular
public class ColaCircular {
    private int[] cola;
    private int frente, finalCola, tamanio, capacidad

    public ColaCircular(int capacidad) {
        this.capacidad = capacidad;
        cola = new int[capacidad];
        frente = 0;
        finalCola = -1;
        tamanio = 0;
    }

    public void enqueue(int valor) {
        if (tamanio == capacidad) {
            System.out.println("Cola llena (Overflow
            return;
        }
        // Avance modular O(1) requerido
        finalCola = (finalCola + 1) % capacidad; //
        cola[finalCola] = valor;
        tamanio++;
    }

    public int dequeue() {
        if (tamanio == 0) {
            System.out.println("Cola vacía (Underflo
            return -1;
```

Demostración en consola que integra el caso del foro y todas las estructuras lineales

```java
Main.java > Main
public class Main {
    Run | Debug
    public static void main(String[] args) {
        System.out.println("=== DEMOSTRACIÓN DE ESTRUCTURAS
        // Simulación del Caso del Foro (Matriz)
        CasoForo cine = new CasoForo(filas: 3, columnas: 3);
        cine.mostrarMapa();
        cine.reservarAsiento(fila: 1, col: 1);
        cine.mostrarMapa();

        // Demostración de Pila y Cola
        PilaLIFO pila = new PilaLIFO(capacidad: 5);
        pila.push(valor: 10);
        System.out.println("Pila pop: " + pila.pop());

        ColaCircular cola = new ColaCircular(capacidad: 3);
        cola.enqueue(valor: 100);
        System.out.println("Cola dequeue: " + cola.dequeue());
    }
```

```
cine.mostrarMapa();

// Demostración de Pila y Cola

Problemas  Terminal  ...  Run: Main

va Cine'; & 'C:\Users\LENOVO\.vscode\extensions\redhat.java-1.58.0-win32-x64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\LENOVO\AppData\Roaming\Code\User\workspaceStorage\13888f6ff1030854ad82f545099240a9\redhat.java\jdt_ws\Java Cine_f36d8eab\bin' 'Main'
=== DEMOSTRACIÓN DE ESTRUCTURAS DE DATOS LINEALES ===
--- MAPA DEL AUDITORIO ---
[Libre] [Libre] [Libre]
[Libre] [Libre] [Libre]
[Libre] [Libre] [Libre]
Asiento 1-1 reservado con éxito.
--- MAPA DEL AUDITORIO ---
[Libre] [Libre] [Libre]
[Libre] [Ocupado] [Libre]
[Libre] [Libre] [Libre]
Pila pop: 10
Cola dequeue: 100
PS C:\Users\LENOVO\Desktop\Java Cine>
```

Código ejecutado

A lo largo del desarrollo de este portafolio, se ha evidenciado que la selección de una estructura de datos lineal impacta directamente en el rendimiento algorítmico y la eficiencia del software:

- Presentan una complejidad temporal de O(1) para el acceso, lectura y actualización directa por medio de índices (filas y columnas). Esto justifica plenamente su elección para el Sistema de Asientos y Mapeo en Tiempo Real para Cine, ya que permite consultar o cambiar el estado de una butaca de forma instantánea, optimizando los tiempos de respuesta en taquilla o plataformas web.
- Su complejidad para la búsqueda de elementos específicos es de O(n), debido a que exigen un recorrido secuencial nodo por nodo a través de las referencias de memoria. Sin embargo, destacan por su flexibilidad dinámica frente a los arreglos estáticos de tamaño fijo.
- La implementación de la cola circular mediante la fórmula de avance modular (rear + 1) % capacidad garantiza operaciones de inserción y extracción en O(1), reutilizando eficientemente las posiciones del arreglo y previniendo errores críticos de desbordamiento (overflow/underflow).